

Project

Ai Image Generator

- Index.html

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>AI Image Generator</title>
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/font-awesome@6.4.0/css/all.min.css">
  <link rel="stylesheet" href="style.css" />
</head>

<body>
  <div class="container">
    <header class="header">
      <div class="logo-wrapper">
        <div class="logo">
          <i class="fa-solid fa-wand-magic-sparkles"></i>
        </div>
        <h1>AI Image Generator</h1>
      </div>

      <button type="button" class="theme-toggle">
        <i class="fa-solid fa-moon"></i>
      </button>
    </header>

    <div class="main-content">
      <form action="#" class="prompt-form">
        <div class="prompt-container">
          <textarea class="prompt-input" placeholder="Describe your imagination in detail..." required
            autofocus></textarea>
          <button type="button" class="prompt-btn">
```

```
<i class="fa-solid fa-dice"></i>
</button>
</div>

<div class="prompt-actions">
  <div class="select-wrapper">
    <select class="custom-select" id="model-select" required>
      <option value="">Select Model</option>
      <option value="black-forest-labs/FLUX.1-dev">FLUX.1-dev </option>
      <option value="black-forest-labs/FLUX.1-schnell"> FLUX.1-schnell </option>

    </select>
  </div>

  <div class="select-wrapper">
    <select class="custom-select" id="count-select" required>
      <option value="" selected disabled>Image Count</option>
      <option value="1">1 Image</option>
      <option value="2">2 Image</option>
      <option value="3">3 Image</option>
      <option value="4">4 Image</option>
    </select>
  </div>

  <div class="select-wrapper">
    <select class="custom-select" id="ratio-select" required>
      <option value="" selected disabled>Aspect Ratio</option>
      <option value="1/1">Square (1:1)</option>
      <option value="16/9">Landscape (16:9)</option>
      <option value="9/16">Portrait (9:16)</option>
    </select>
  </div>

  <button type="submit" class="generate-btn">
    <i class="fa-solid fa-wand-sparkles"></i>
    Generate
  </button>
</div>

<div class="gallery-grid">
  <div class="img-card loading">
    <div class="status-container">
      <div class="spinner"></div>
      <i class="fa-solid"></i>
      <p class="status-text">Generating...</p>
```

```

        </div>
        
        <div class="img-overlay">
            <button class="image-download-btn">
                <i class="fa-solid fa-download"></i>
            </button>
        </div>
    </div>
    <div class="img-card">
        
        <div class="img-overlay">
            <button class="image-download-btn">
                <i class="fa-solid fa-download"></i>
            </button>
        </div>
    </div>
</div>
</form>
</div>
</div>

```

```

    <script src="script.js"></script>
</body>

```

```
</html>
```

• Style.css

```

@import
url('https://fonts.googleapis.com/css2?family=Inter:ital,opsz,wght@0,14..32,100..900;1,14..32,100..900&display=swap');

```

```

* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
    font-family: "Inter", sans-serif;
}

```

```

:root {
    --color-primary: #5C56E1;
    --color-primary-dark: #5b21b6;
    --color-accent: #8B5CF6;
    --color-card: #FFFFFF;
    --color-input: #F1F1FF;
    --color-text: #09090E;
    --color-placeholder: #5C5A87;
    --color-border: #D4D4ED;
}

```

```
--color-gradient: linear-gradient(135deg, #5C56E1, #8B5CF6);
}

/* Custom Scrollbar */
::-webkit-scrollbar {
  width: 10px;
}

::-webkit-scrollbar-track {
  background: var(--color-input);
}

::-webkit-scrollbar-thumb {
  background: var(--color-placeholder);
  border-radius: 5px;
}

::-webkit-scrollbar-thumb:hover {
  background: var(--color-primary);
}

body.dark-theme {
  --color-card: #1E293B;
  --color-input: #141B2D;
  --color-text: #F3F4F6;
  --color-placeholder: #A3B6DC;
  --color-border: #334155;

  background: #0F172A;
  /* Deep slate background */
  background-image: none;
  /* Removed complex gradient for clarity */
}

body {
  display: flex;
  align-items: center;
  justify-content: center;
  min-height: 100vh;
  padding: 15px;
  color: var(--color-text);
  background: linear-gradient(#E9E9FF, #CBC7FF);
}

.container {
  width: 900px;
```

```
padding: 32px;
position: relative;
border-radius: 23px;
overflow: hidden;
background: var(--color-card);
box-shadow: 0 10px 20px 0 rgba(0, 0, 0, 0.1);
}
```

```
body.dark-theme .container {
border: 1px solid var(--color-border);
}
```

```
.container::before {
content: "";
position: absolute;
top: 0;
left: 0;
width: 100%;
height: 5px;
background: var(--color-gradient);
}
```

```
.header {
display: flex;
align-items: center;
justify-content: space-between;
}
```

```
.header .logo-wrapper {
display: flex;
gap: 18px;
align-items: center;
}
```

```
.header .logo-wrapper .logo {
height: 55px;
width: 56px;
display: flex;
color: #fff;
font-size: 1.35rem;
flex-shrink: 0;
border-radius: 15px;
align-items: center;
justify-content: center;
background: var(--color-gradient);
}
```

```
.header .logo-wrapper h1 {  
  font-size: 1.9rem;  
  font-weight: 700;  
}
```

```
.header .theme-toggle {  
  height: 43px;  
  width: 43px;  
  border-radius: 50%;  
  font-size: 1.05rem;  
  cursor: pointer;  
  display: flex;  
  color: var(--color-placeholder);  
  align-items: center;  
  justify-content: center;  
  background: var(--color-input);  
  border: 1px solid var(--color-border);  
}
```

```
.header .theme-toggle:hover {  
  color: var(--color-primary);  
  transform: translateY(-2px);  
  box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1);  
}
```

```
.header .theme-toggle:active {  
  transform: translateY(0);  
  box-shadow: none;  
}
```

```
.main-content {  
  margin: 35px 0 5px;  
}
```

```
.main-content .prompt-input {  
  width: 100%;  
  resize: vertical;  
  line-height: 1.6;  
  font-size: 1.05rem;  
  min-height: 120px;  
  padding: 16px 20px;  
  border-radius: 15px;  
  color: var(--color-text);  
  background: var(--color-input);  
  border: 1px solid var(--color-border);  
  transition: all 0.3s ease;
```

```
}

.main-content .prompt-input::placeholder {
  color: var(--color-placeholder);
}

.main-content .prompt-input:focus {
  outline: none;
  background: #FFFFFF;
  border-color: var(--color-accent);
  box-shadow: 0 0 4px rgba(139, 92, 246, 0.15);
}

.prompt-container {
  position: relative;
}

.prompt-container .prompt-btn {
  position: absolute;
  right: 15px;
  bottom: 15px;
  height: 35px;
  width: 35px;
  border: none;
  color: #fff;
  font-size: 0.75rem;
  border-radius: 50%;
  opacity: 0.8;
  cursor: pointer;
  background: var(--color-gradient);
  transition: all 0.3s ease;
}

.prompt-container .prompt-btn:hover {
  opacity: 1;
  transform: translateY(-2px);
  box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1);
}

.prompt-container .prompt-btn:active {
  transform: translateY(0);
  box-shadow: none;
}

.main-content .prompt-actions {
  display: grid;
  gap: 14px;
```

```
    grid-template-columns: 1.2fr 1fr 1.1fr 1fr;
}

.prompt-actions .select-wrapper {
    position: relative;
}

.prompt-actions .select-wrapper::after {
    content: "";
    font-family: "Font Awesome 6 Free";
    font-weight: 900;
    position: absolute;
    right: 20px;
    top: 50%;
    font-size: 0.9rem;
    padding-left: 7px;
    pointer-events: none;
    background: var(--color-input);
    color: var(--color-placeholder);
    transform: translateY(-50%);
}

.prompt-actions :where(.custom-select, .generate-btn) {
    cursor: pointer;
    padding: 12px 20px;
    font-size: 1rem;
    border-radius: 10px;
    background: var(--color-input);
    border: 1px solid var(--color-border);
    transition: all 0.3s ease;
}

.prompt-actions .custom-select {
    width: 100%;
    outline: none;
    height: 100%;
    appearance: none;
    color: var(--color-text);
}

.prompt-actions .custom-select:is(:focus, :hover) {
    background: #FFFFFF;
    color: #000000;
    border-color: var(--color-accent);
}
```



```
    box-shadow: 0 0 0 3px rgba(139, 92, 246, 0.1);
  }

.prompt-actions .generate-btn {
  display: flex;
  gap: 12px;
  margin-left: auto;
  font-weight: 500;
  align-items: center;
  justify-content: center;
  padding: 12px 25px;
  border: none;
  color: #fff;
  background: var(--color-gradient);
}

.prompt-actions .generate-btn:hover {
  transform: translateY(-2px);
  box-shadow: 0 4px 10px rgba(109, 40, 217, 0.3);
}

.prompt-actions .generate-btn:active {
  transform: translateY(0);
  box-shadow: none;
}

.prompt-actions .generate-btn:disabled {
  opacity: 0.5;
  cursor: not-allowed;
  pointer-events: none;
  background: var(--color-placeholder);
  color: var(--color-text);
}

.main-content .gallery-grid:has(.img-card) {
  margin-top: 30px;
}

.main-content .gallery-grid {
  display: grid;
  gap: 15px;
  margin-top: 30px;
  grid-template-columns: repeat(auto-fill, minmax(180px, 1fr));
}

.gallery-grid .img-card {
  display: flex;
```

```
flex-direction: column;
align-items: center;
justify-content: center;
position: relative;
overflow: hidden;
aspect-ratio: 1;
border-radius: 16px;
background: var(--color-input);
border: 1px solid var(--color-border);
transition: all 0.5s ease;
}

.gallery-grid .img-card:not(.loading, .error):hover {
  transform: translateY(-5px);
  box-shadow: 0 10px 15px -3px rgba(0, 0, 0, 0.1);
}

.gallery-grid .img-card .result-img {
  width: 100%;
  height: 100%;
  object-fit: cover;
}

.gallery-grid .img-card:is(.loading, .error) :is(.result-img, .img-overlay) {
  display: none;
}

.gallery-grid .img-card .img-overlay {
  position: absolute;
  bottom: 0;
  left: 0;
  right: 0;
  padding: 20px;
  display: flex;
  opacity: 0;
  pointer-events: none;
  justify-content: flex-end;
  background: linear-gradient(transparent, rgba(0, 0, 0, 0.8));
  transition: all 0.3s ease;
}

.gallery-grid .img-card:hover .img-overlay {
  opacity: 1;
  pointer-events: auto;
}
```

```
.gallery-grid .img-card .img-overlay .img-download-btn {
  height: 45px;
  width: 45px;
  color: #fff;
  backdrop-filter: blur(5px);
  border-radius: 50%;
  border: none;
  cursor: pointer;
  background: rgba(255, 255, 255, 0.25);
  transition: all 0.3s ease;
}

.gallery-grid .img-card .img-overlay .img-download-btn:hover {
  transform: scale(1.05);
  background: rgba(255, 255, 255, 0.4);
}

.gallery-grid .img-card .status-container {
  padding: 15px;
  display: none;
  gap: 13px;
  flex-direction: column;
  align-items: center;
}

.gallery-grid .img-card:where(.loading, .error) .status-container {
  display: flex;
}

.gallery-grid .img-card.loading .status-container i,
.gallery-grid .img-card.error .spinner {
  display: none;
}

.gallery-grid .img-card.error .status-container i {
  font-size: 1.7rem;
  color: #ef4444;
}

.gallery-grid .img-card.loading .spinner {
  width: 35px;
  height: 35px;
  border-radius: 50%;
  border: 3px solid var(--color-border);
}
```

```
border-top-color: var(--color-primary);
animation: spin 1s linear infinite;
}
```

```
@keyframes spin {
  to {
    transform: rotate(360deg);
  }
}
```

```
.gallery-grid .img-card .status-text {
  font-size: 0.85rem;
  text-align: center;
  color: var(--color-placeholder);
}
```

- **Test.png**



- **Script.js**

```
const themeToggle = document.querySelector(".theme-toggle");
const promptForm = document.querySelector(".prompt-form");
const promptInput = document.querySelector(".prompt-input");
const modelselect = document.getElementById("model-select");
const countselect = document.getElementById("count-select");
const ratiosselect = document.getElementById("ratio-select");
const promptBtn = document.querySelector(".prompt-btn");
const generateBtn = document.querySelector(".generate-btn");
const gridGallery = document.querySelector(".gallery-grid");
```

```
const API_KEY = "hf_aNizIbfhFHYOQGpcPgTQYAZiRRHSXUENS";
```

```
const examplePrompts = [
  "A magic forest with glowing plants and fairy homes among giant mushrooms",
  "An old steampunk airship floating through golden clouds at sunset",
  "A future Mars colony with glass domes and gardens against red mountains",
```

```

"A dragon sleeping on gold coins in a crystal cave",
"An underwater kingdom with merpeople and glowing coral buildings",
"A floating island with waterfalls pouring into clouds below",
"A witch's cottage in fall with magic herbs in the garden",
"A robot painting in a sunny studio with art supplies around it",
"A magical library with floating glowing books and spiral staircases",
"A Japanese shrine during cherry blossom season with lanterns and misty mountains"
];

```

```

(() => {
  const savedTheme = localStorage.getItem("theme");
  const systemPrefersDark = window.matchMedia("(prefers-color-scheme: dark)").matches;

  const isDarkTheme = savedTheme === "dark" || !savedTheme;
  document.body.classList.toggle("dark-theme", isDarkTheme);
  themeToggle.querySelector("i").className = isDarkTheme ? "fa-solid fa-sun" : "fa-solid fa-moon";
})();

```

```

const toggleTheme = () => {
  const isDarkTheme = document.body.classList.toggle("dark-theme");
  localStorage.setItem("theme", isDarkTheme ? "dark" : "light");
  themeToggle.querySelector("i").className = isDarkTheme ? "fa-solid fa-sun" : "fa-solid fa-moon";
};

```

```

const getImageDimensions = (aspectRatio) => {
  const [width, height] = aspectRatio.split("/").map(Number);
  // Base size 512, scaled by ratio (simplified logic)
  if (aspectRatio === "1/1") return { width: 1024, height: 1024 };
  if (aspectRatio === "16/9") return { width: 1024, height: 576 };
  if (aspectRatio === "9/16") return { width: 576, height: 1024 };
  return { width: 1024, height: 1024 };
}

const updateImageCard = (imgIndex, imgUrl) => {
  const imgCard = document.getElementById(`img-card-${imgIndex}`);
  if (!imgCard) return;
  imgCard.classList.remove("loading");
  imgCard.innerHTML = `
    <div class="img-overlay">
      <a href="${imgUrl}" class="img-download-btn" download="${Date.now()}.png">
        <i class="fa-solid fa-download"></i>
      </a>
    </div>`;
}

const generateImages = async (selectModel, imageCount, aspectRatio, promptText) => {

```

```

const MODEL_URL = `https://router.huggingface.co/hf-inference/models/${selectModel}`;
const { width, height } = getImageDimensions(aspectRatio);

const imagePromises = Array.from({ length: imageCount }, async (_, i) => {
  try {
    const response = await fetch(MODEL_URL, {
      headers: {
        Authorization: `Bearer ${API_KEY}`,
        "Content-Type": "application/json",
      },
      method: "POST",
      body: JSON.stringify({
        inputs: promptText,
        parameters: { width, height },
        options: { wait_for_model: true, user_cache: false },
      }),
    });
    if (!response.ok) throw new Error((await response.json())?.error);
    const result = await response.blob();
    updateImageCard(i, URL.createObjectURL(result));
  } catch (error) {
    console.log(error);
    const imgCard = document.getElementById(`img-card-${i}`);
    if (imgCard) {
      imgCard.classList.remove("loading");
      imgCard.classList.add("error");
      imgCard.querySelector(".status-text").innerText = error.message;
    }
  }
});
await Promise.allSettled(imagePromises);
};

const createImageCards = (selectModel, imageCount, aspectRatio, promptText) => {
  gridGallery.innerHTML = "";

  for (let i = 0; i < imageCount; i++) {
    gridGallery.innerHTML += `<div class="img-card loading" id="img-card-${i}" style="aspect-
ratio: ${aspectRatio}">
      <div class="status-container">
        <div class="spinner"></div>
        <i class="fa-solid"></i>
        <p class="status-text">Generating...</p>
      </div>
    </div>`;
  }
}

```

```
    generateImages(selectModel, imageCount, aspectRatio, promptText);
  }

const handleFormSubmit = (e) => {
  e.preventDefault();

  const selectModel = modelselect.value;
  const imageCount = parseInt(countselect.value) || 1;
  const aspectRatio = ratiosselect.value || "1/1";
  const promptText = promptInput.value.trim();

  console.log(selectModel, imageCount, aspectRatio, promptText);
  createImageCards(selectModel, imageCount, aspectRatio, promptText);
}

promptForm.addEventListener("submit", handleFormSubmit);

const updateGenerateBtnState = () => {
  generateBtn.disabled = promptInput.value.trim() === "";
};

updateGenerateBtnState();
promptInput.addEventListener("input", updateGenerateBtnState);

promptBtn.addEventListener("click", () => {
  const prompt = examplePrompts[Math.floor(Math.random() * examplePrompts.length)];
  promptInput.value = prompt;
  updateGenerateBtnState();
  promptInput.focus();
})

themeToggle.addEventListener("click", toggleTheme);
```

- **Result :**

